

Retail Store Analytics — README

Official technical documentation, system architecture flow, local setup guides, and project business assumptions for Neeman's Retail Intelligence.

1. Problem Statement

Modern Direct-to-Consumer (D2C) brands like Neeman's deal with complex supply chain and inventory flows across multiple retail formats (flagships, mall stores, outlets). Store managers and supply chain coordinators need to make fast, daily decisions regarding:

- Restocking fast-moving bestsellers to prevent stockouts (which represent lost revenue).
- Identifying stagnant/dead stock to run targeted discounts or transfer units to high-performing locations.
- Ranking store operational efficiency.

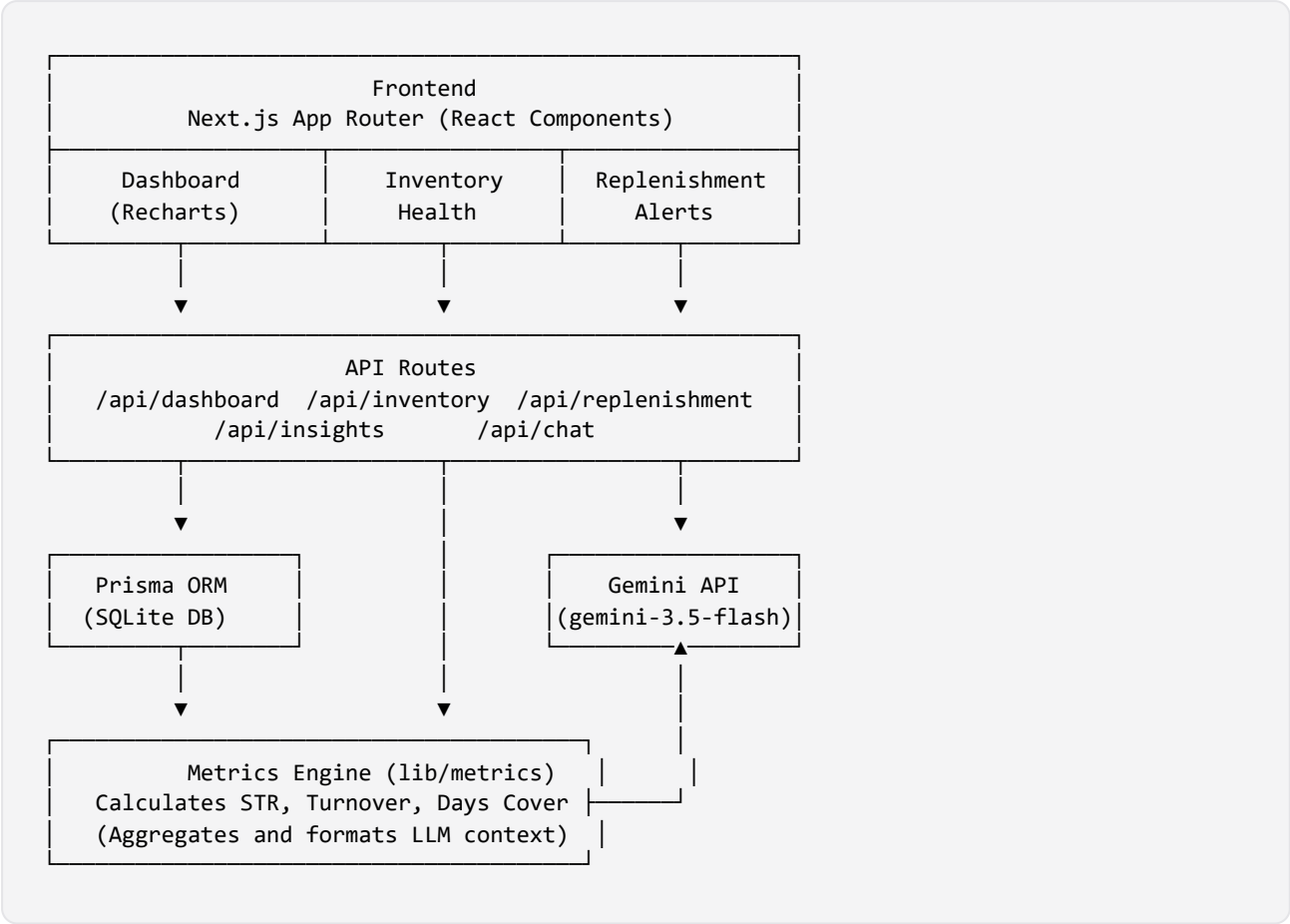
Relying on manual spreadsheet audits is slow and error-prone. This application solves this by consolidating store sales and inventory logs, calculating deterministic retail metrics, and layer-integrating a Large Language Model (LLM) to deliver plain-English recommendations and a Q&A agent.

2. Solution Overview

The Retail Store Analytics dashboard provides a unified dashboard featuring:

1. **KPI Dashboard:** A high-level overview of total revenue, units sold, average sell-through %, and alerts.
2. **Interactive Chart:** Recharts area chart showing daily units sold over 75 days, filterable by store and category.
3. **Store Leaderboard:** Ranked store list based on a composite performance score.
4. **Inventory Health Tab:** Filterable grid displaying every SKU × Store status (Healthy, Low Stock, Overstock, Dead Stock) and days of cover.
5. **Replenishment alerts page:** restock recommendations with sliding controls that recalculate suggested reorder quantities in real-time.
6. **AI Insights & Q&A Chat:** Contextual insights and conversational Q&A powered by the Gemini 3.5 Flash model.

3. Architecture & Data Flow



4. AI Integration & Cache Policy

- **Model Used:** `gemini-3.5-flash` via the `@google/genai` SDK.
- **Data Privacy & Cost Reduction:** Instead of sending raw, granular transaction logs (thousands of rows) to the LLM, the **Metrics Engine** calculates aggregates (KPIs, top/bottom performing stores, low cover alerts, stagnant SKUs) first. This summarized metadata is sent inside the prompt.
- **Cache Strategy:** AI Insights are generated and stored in a local cache file (`dev_insights_cache.json`). The application retrieves insights from the cache on page load. Users can trigger a fresh analysis by clicking the **Regenerate** button, preventing unnecessary API expenses.

5. Local Setup Instructions

Prerequisites

- Node.js: v18.0.0 or higher (v24.14.1 used during development).
- npm: v9.0.0 or higher.

Installation Steps

1. Clone/Move into project folder:

```
cd retail-store-analytics
```

2. Install dependencies:

```
npm install --legacy-peer-deps
```

3. Configure Environment Variables:

Create a `.env` file in the root of the project and add your Gemini API Key:

```
DATABASE_URL="file:./dev.db"  
GEMINI_API_KEY="your_gemini_api_key_here"
```

4. Initialize Database and Run Migrations:

```
npx prisma migrate dev --name init
```

5. Seed the Database:

Populates 75 days of transaction history (11,250 records) for 6 stores and 25 products:

```
node prisma/seed.js
```

6. Start the Development Server:

```
npm run dev
```

Open `http://localhost:3000` in your web browser.

6. Assumptions Made

1. **Transaction Timeline:** Sales and inventory values are seeded for a fixed 75-day timeline (from June 5, 2026 to August 18, 2026). The "Current Stock" is defined as the closing stock of the most recent date in the database.
2. **Sell-Through Rate Formula:** Calculated over the entire 75-day period as
$$\frac{\text{units_sold}}{(\text{initial_opening_stock} + \text{units_received})} * 100$$
.
3. **Inventory Turnover:** Calculated as
$$\frac{\text{total_units_sold}}{\text{average_stock}}$$
 over the timeline, where `average_stock` represents the average of daily opening stocks for all SKUs combined.
4. **Days of Stock Cover:** Calculated as
$$\frac{\text{current_stock}}{\text{trailing_7_day_sales_velocity}}$$
. If a SKU has zero sales over the trailing 7 days, its cover is set to `999` days (or `0` if current stock is also `0`).
5. **Dead Stock Warning:** Triggered if a SKU has stock on hand but has generated `0` sales in the trailing 30 days.
6. **Store Performance Score Weights:** Weighted composite index of Revenue (50%), Sell-Through % (30%), and Turnover (20%), normalized out of 100 relative to the highest-performing store.

7. Deployment Instructions

GitHub Repository Push

1. Create a new empty repository on GitHub. Do **not** initialize it with a README, `.gitignore`, or License.
2. Add the remote origin URL, set the branch to `main`, and push your code:

```
git remote add origin <your-github-repo-url>
git branch -M main
git push -u origin main
```

Render Deployment (Web Service)

1. Create a new **Web Service** on Render.
2. Connect your GitHub repository.
3. Configure the following deployment parameters:
 - Runtime: `Node`
 - Build Command: `npm install --legacy-peer-deps && npx prisma generate && node prisma/seed.js && npm run build`
 - Start Command: `npm run start`

4. Add the following **Environment Variables** in Render's "Environment" tab:

- `DATABASE_URL` : `file:./dev.db`
- `GEMINI_API_KEY` : `your_gemini_api_key_here` (e.g. the AQ. auth key)

5. Click **Deploy Web Service**. Render will install packages, compile the database client, seed the transaction history, build the production bundle, and run it. Custom assets and database states are safely built directly into the container.